

Unit -4

Syllabus

Advance exception Handling, collection frameworks -
list, set & map, Enhanced for loop and iterators,
I/o, buffer, file stream.

Java Collection framework

① List interface

* List is the sub-interface of java collection framework, it contains indexed base methods to insert, update, delete and search elements.

* it can have duplicate elements also, we can also store null elements in it

* The list preserves the insertion order and the list is available in java.util package.

WAP to demonstrate Implementing Array list

```
import java.util.*;
```

```
class list demo
```

```
{
```

```
    public static void main (String args [])
```

```
    {
```

```
        List <String> l = new ArrayList <String>();
```

```

l.add("mango");
l.add("apple");
l.add("orange");
l.add("mango");
for (String i : l)
{
    System.out.println(i);
}
}
}

```

O/P:

mango
apple
orange
mango

Linked List:

A linked list is a part of java collection framework that store the elements as nodes. Each node pointing to the next node in the sequence.

Example:

```

import java.util.*;
class Listdemo
{
    public static void main (String args[])
    {
        List<String> l = new LinkedList<String>;
    }
}

```

```

l.add("mango");
l.add("apple");
l.add("orange");
l.add("mango");
for (i: l)
{
    System.out.println(i);
}
}

```

o/p:

mango
apple
orange
mango

Set < Hash set
Tree set

Set:

* The set follows the unordered way for storing the elements, it is available in java util package

* In set the duplicate elements will be ignored, and it will not be print in the final output

Eg: for hash set

```
import java.util.*;  
class setdemo
```

```
{  
    public static void main (String args [])
```

```
{
```

```
    Set <String> s = new HashSet <String>;
```

```
    s.add("mango");
```

```
    s.add("apple");
```

```
    s.add("orange");
```

```
    s.add("mango");
```

```
    for (String i: s)
```

```
    {
```

```
        System.out.println(i);
```

```
    }
```

```
}
```

```
}
```

O/p:

mango

apple

orange

Tree set:

uses the

The tree set is a sorted collection of data and it uses the binary search tree internally red, black tree for storage.

Eg: for TreeSet

```
import java.util.*;  
class setdemo
```

```
{  
    public static void main (String args[])  
    {  
        Set <String> s = new TreeSet <String>;  
        s.add("Porsche");  
        s.add("BMW");  
        s.add("Jaguar");  
        s.add("BMW");  
        for (String i: s)  
        {  
            System.out.println(i);  
        }  
    }  
}
```

O/P:

Porsche
BMW
Jaguar

Maps :

- * Java Map interface is available in util package
- * The map represents the mapping between key and value, more specifically java map can store pair of keys and values, each key is linked to specific value.
- * Once stored in a Map, we can easily retrieve the values just by using keys

Map - Hash Map

```
import java.util.*;
class mapdemo
{
    public static void main (String args [])
    {
        Map <int, String> m = new HashMap <int, String>
        ();
        m.put (100, "Ajay");
        m.put (101, "Akil");
        m.put (102, "Akila");
        for (MapEntry m1: map.entrySet())
        {
            System.out.println (m1.getKey() + m1.getValue())
        }
    }
}
```

O/P:

100	Ajay
101	Akil
102	Akila

Eg: for Tree Map

Tree Map:

The Tree Map stores the values as sorted collection of data and it uses balanced binary search tree that is red-black tree for storage.

Eg:

```
import java.util.*;
```

```
class mapdemo
```

```
{
```

```
public static void main (String args[])
```

```
{
```

```
Map<int, String> m = new TreeMap<int, String>();
```

```
m.put(100, "Ajay");
```

```
m.put(101, "Akil");
```

```
m.put(102, "Akila");
```

```
for (MapEntry m1 : m.entrySet())
```

```
{
```

```
System.out.println(m1.getKey() + " " + m1.getValue());
```

```
}
```

```
}
```

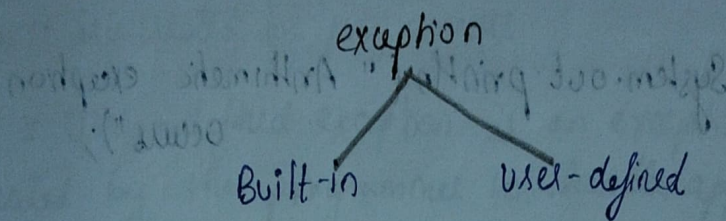
O/P:

100 Ajay

101 Akil

102 Akila

Advanced exception Handling



1) Arithmetic

2) divide by zero

3) Array index out of bound

The exception are classified into two types: Built-in, User-defined exceptions

1. Built-in exception

A built-in exception is an exception that is already provided by java, these exceptions are used to handle common run-time errors.

Eg:

```
import java.io.*;
```

```
class exceptiondemo
```

```
{
```

```
    public static void main (String args[])
```

```
    {
```

```
        try
```

```
        {
```

```
            int a[5] = new int[5];
```

```
            a[6] = 30/0;
```

```
        }
```

catch (ArithmeticException)

{

System.out.println("Arithmetic exception occurs");

}

catch (ArrayIndexOutOfBoundsException)

{

System.out.println("Array exception occurs");

}

catch (Exception)

{

System.out.println("exception occurs");

}

finally {}

{

System.out.println("Rest of the code");

}

}

}

O/P:

Arithmetic exception occurs

Array exception occurs

Rest of the code.

2. User-defined exception

- * It is created by programmers

- * User-defined exception is an exception created by the programmer to handle application specific errors

Eg:

```
class AgeException extends Exception
```

```
{
```

```
}
```

```
class userDefinedException
```

```
{
```

```
    public static void (String args [])
```

```
    {
```

```
        int age = 15;
```

```
        try
```

```
        {
```

```
            if (age < 18)
```

```
            {
```

```
                throw new AgeException();
```

```
            }
```

```
        else
```

```
        {
```

```
            System.out.println ("Eligible to vote");
```

```
        }
```

```
    }
```

```
    catch (AgeException e)
```

```
{  
    System.out.println("Not eligible");  
}
```

```
}  
}
```

O/p:

Not eligible

Unit-4 Continuation

File I/O - (Input-Output)

- * File I/O refers to reading data from files and writing data into the files
- * Java provides classes in the java.io package to perform file operations
- * The basic file I/O flow:

For writing $\rightarrow$ output stream & file

For reading $\rightarrow$ input stream & file

File reader:

The file reader is used to read a character data from a file

Syntax:

```
FileReader reader = new FileReader("data.txt");
```

Program:

```
import java.io.*;
```

```
class FileReaderDemo
```

```
{
```

```
    public static void main (String args[])
```

```
{
```

```
    FileReader reader = new FileReader("intro.txt");
```

```
int ch;
```

```
while ((ch = reader.read()) != -1)
```

```
{
```

```
    System.out.println((char)ch)
```

```
}
```

```
reader.close();
```

```
}
```

```
} ; ("txt.txt")
```

O/p

Hello welcome

info.txt
Hello, welcome

[The file reader is used for reading text files it reads the data character by character]

lead()

[The lead() function return -1, when the end of the file is reached]

close()

[The close() is used to close the file]

File Writer

* File writer is used to write a character data to a file

* It is used for writing text to files.

* If the file does not exist, the java can create it.

Write()

The Write() is used to write the data to the file

close()

close() is used to save and close the file.

Syntax:

```
FileWriter writer = new FileWriter("output.txt");
```

Program

```
import java.io.*;
```

```
class FileWriterDemo
```

```
{
```

```
    public static void main(String args[])
```

```
{
```

```
        FileWriter writer = new FileWriter("Output.txt");
```

```
        writer.write("Welcome to my program");
```

```
        writer.close();
```

```
    }
```

```
}
```

O/P

output.txt
Welcome to my program

The output will be visible in the output.txt file.

Buffered Reader

The Buffered Reader is used to read the text efficiently especially line by line. It is usually used together with file reader class.

Syntax:

```
BufferedReader br = new BufferedReader (new FileReader  
(or) ("data.txt"));
```

```
FileReader r = new FileReader ("data.txt");
```

```
BufferedReader br = new BufferedReader (r);
```

Program

```
import java.io.*;
```

```
class bufferedemo
```

```
{  
    public static void main (String args [])
```

```
{
```

```
    BufferedReader br = new BufferedReader (new FileReader  
("data.txt"));
```

```
    String line;
```

```
    while ((line = br.readLine()) != null)
```

```
{
```

```
        System.out.println (line);
```

```
}
```

```
    br.close();
```

```
}
```

```
}
```

o/p:-

readLine()

The readLine() is used to read one complete line from the file. When there are no more lines, it returns null.

BufferedWriter

The BufferedWriter is used to write the text efficiently using "buffer". It is commonly used with FileWriter.

Syntax:-

```
BufferedWriter bw = new BufferedWriter(new FileWriter("output.txt"));
```

or

```
FileWriter w = new FileWriter("output.txt");
```

```
BufferedWriter bw = new BufferedWriter(w);
```

program:

```
import java.io.*;

class bufferedWriterDemo
{
    public static void main(String args[])
    {
```

```

BufferedWriter bw = new BufferedWriter(new FileWriter(
    "output.txt"));
bw.write("Welcome to my program");
bw.close();

```

}

have not been, introduction to AWT and
 Swing, layout manager, event delegation
 model, handling our events, Action
 listener, mouse listener, Building basic
 forms and interfaces with buttons
 labels, text fields

Swing

Swing is a part of Java Foundation classes (JFC)

that is used to create graphical user interface with
 Applications

for GUI

It is built on AWT (Abstract Window Toolkit)

It is more powerful, flexible and lightweight

Swing provides a rich set of components

like buttons, text fields, tables etc.